

0.0.2. La fonction `texte()` — écrire un mot

`#include <texte>`, puis `texte(colonne, ligne, "MOT")` : le mot s'écrit à cette place, une lettre par case.

1. Ce qu'il faut comprendre

Une fonction, une étape. Avant d'employer une fonction de la console pour la première fois, le parcours la présente seule : la ligne `#include` qui la rend disponible, ses arguments, ce qu'elle coûte, et un essai. Ici, `texte()`, la première de toutes.

Ce qu'elle fait : `texte(colonne, ligne, "MOT")` écrit des lettres à l'écran, une par case, de gauche à droite. L'écran fait **20 colonnes** (0 à 19) et **18 lignes** (0 à 17).

La ligne à écrire : `#include <texte>`. Sans elle, le compilateur refuse `texte()` et dit quelle ligne écrire.

Ses arguments : la **colonne** de la première lettre, la **ligne**, puis le **texte entre guillemets** : des lettres, des chiffres, des espaces. La colonne et la ligne peuvent être des variables.

Ce qu'elle coûte : environ **150 octets** la première fois (la mise en place de l'écran, la routine qui écrit, les lettres employées), puis une dizaine d'octets par appel.

Essaie : écris ton prénom en (0, 17), tout en bas à gauche.

On retrouve `texte` dans 302 leçons

[0.0.1](#), [0.1](#), [0.1.1](#), [0.1.2](#), [0.3](#), [0.3.1](#), [0.3.2](#), [0.4](#), [0.4.1](#), [0.4.2](#), [0.79](#), [0.79.1](#), [0.81](#), [0.81.1](#), [0.81.3](#), [0.82.1](#), [0.83](#), [0.83.2](#), [0.84](#), [0.88](#), [0.88.1](#), [0.88.2](#), [0.89.2](#), [0.90](#), [0.90.1](#), [0.90.2](#), [0.90.4](#), [0.90.5](#), [0.90.6](#), [0.90.7](#), [0.90.8](#), [0.91](#), [0.91.1](#), [0.91.2](#), [0.91.3](#), [0.91.4](#), [0.91.5](#), [0.91.6](#), [0.91.7](#), [0.91.8](#), [0.91.9](#), [0.92](#), [0.92.1](#), [0.92.2](#), [0.93.3](#), [0.93.4](#), [0.93.5](#), [0.93.6](#), [0.93.7](#), [0.93.8](#), [0.93.9](#), [0.94](#), [0.94.1](#), [0.95](#), [0.96](#), [0.96.1](#), [0.97](#), [0.97.1](#), [0.98](#), [0.98.1](#), [0.99](#), [0.99.1](#), [0.100](#), [0.100.1](#), [0.101](#), [0.101.1](#), [0.102](#), [0.102.1](#), [0.103](#), [0.104](#), [0.105](#), [0.105.1](#), [0.106](#), [0.107](#), [0.107.1](#), [0.108](#), [0.1](#), [1](#), [1.1](#), [1.2](#), [1.3](#), [1.4](#), [1.5](#), [1.6](#), [1.8](#), [1.9](#), [1.10](#), [1.11](#), [1.12](#), [2](#), [2.1](#), [2.2](#), [2.3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [8.1](#), [9](#), [10](#), [11](#), [12](#), [12.1](#), [13](#), [14](#), [16](#), [17](#), [18](#), [19](#), [20](#), [20.1](#), [21](#), [22](#), [23](#), [24](#), [24.1](#), [25](#), [26](#), [27](#), [28](#), [29](#), [30](#), [31](#), [32](#), [33](#), [33.1](#), [34](#), [35](#), [35.12](#), [35.15](#), [35.18](#), [36](#), [37](#), [38](#), [39](#), [41](#), [43](#), [43.1](#), [44](#), [45](#), [46](#), [47](#), [52.1](#), [53](#), [54](#), [55](#), [56](#), [57](#), [58](#), [59](#), [59.1](#), [60](#), [65](#), [66](#), [66.1](#), [67](#), [68](#), [69](#), [70](#), [71](#), [71.1](#), [72](#), [73](#), [75](#), [77](#), [78](#), [78.1](#), [79](#), [80](#), [81](#), [82](#), [82.1](#), [83](#), [84](#), [85](#), [86](#), [90](#), [90.1](#), [91](#), [92](#), [93](#), [94](#), [94.1](#), [95](#), [96](#), [97](#), [98](#), [99](#), [100](#), [100.1](#), [101](#), [102](#), [103](#), [104](#), [105](#), [106](#), [106.1](#), [107](#), [108](#), [109](#), [110](#), [110.3](#), [110.4](#), [110.5](#), [110.6](#), [110.7](#), [110.8](#), [110.9](#), [110.11](#), [110.13](#), [110.14](#), [110.15](#), [111](#), [112](#), [113](#), [113.1](#), [114](#), [115](#), [116](#), [117](#), [117.1](#), [118](#), [119](#), [120](#), [120.1](#), [121](#), [122](#), [123](#), [123.1](#), [124](#), [125.1](#), [126](#), [127](#), [128](#), [129](#), [133](#), [134](#), [134.1](#), [135](#), [136](#), [138](#), [139](#), [141](#), [143](#), [143.1](#), [2.01](#), [2.02](#), [2.03](#), [2.04](#), [2.05](#), [2.06](#), [2.07](#), [2.08](#), [2.09](#), [2.10](#), [2.11](#), [2.12](#), [2.13](#), [2.14](#), [2.15](#), [2.16](#), [2.17](#), [2.18](#), [2.19](#), [2.20](#), [2.21](#), [2.22](#), [2.23](#), [2.24](#), [2.25](#), [2.26](#), [2.27](#), [2.28](#), [2.29](#), [2.30](#), [2.31](#), [2.32](#), [2.33](#), [2.34](#), [2.35](#), [2.36](#), [2.37](#), [2.38](#), [2.39](#), [3.01](#), [3.02](#), [3.03](#), [3.04](#), [3.05](#), [3.06](#), [3.07](#), [3.08](#), [3.09](#), [3.10](#), [3.11](#), [3.12](#), [3.13](#), [3.14](#), [3.15](#), [3.16](#), [3.17](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **425 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	29 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte	– attend de pouvoir écrire une case
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	85 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	160 o	← CopierTuiles	– 10 lettres sur 43
"BONJOUR"	7 o	← texte()	– un texte gravé
"EN HAUT"	7 o	← texte()	– un texte gravé

Lettres gravées : ABEHJNORTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte() × 2	22 o sur place, dans main()	+ EcrireTexte (10 o)
image() × 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <texte> : écrire un mot ----
// texte(colonne, ligne, "MOT") écrit le mot à cette place, une lettre par case.

#include <texte> // texte() : sans cette ligne, le compilateur la refuse
```

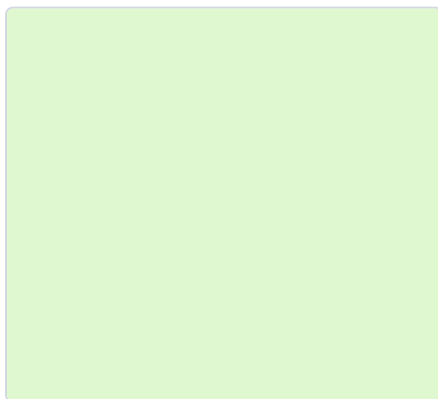
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(4, 8, "BONJOUR"); // colonne 4, ligne 8 : B en (4, 8), O en (5, 8)...
    texte(0, 0, "EN HAUT"); // la case (0, 0) est en haut à gauche

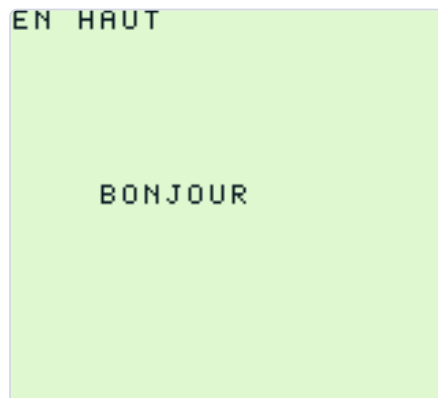
    while (true) {           // la boucle du jeu : rien d'autre à faire
        image();             // image() est native : elle ne s'inclut pas
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

BONJOUR au milieu de l'écran, EN HAUT dans le coin en haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ BONJOUR est en (4, 8) (« BONJOUR »)

✓ EN HAUT est en (0, 0)

Ensuite : **0.1. Afficher la lettre A** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++